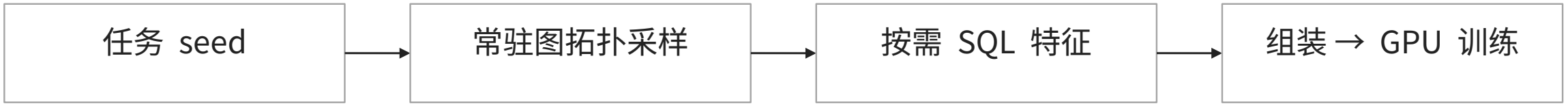
RelConnector 优化演进：在有限内存下完成关系图训练

核心问题：业务特征无法全量放进内存，如何减少在线训练的读取、编码和等待？

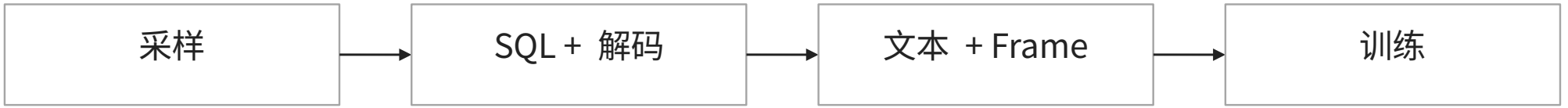


版本	基本方式	优化重点	正式比较角色
vanilla	按采样节点读取特征	已有基础 SQL / raw cache	未引入后续优化的基线
v1	在线读取 + 分层复用	去重、文本 / encoded 缓存	单进程优化版本
v2	采样 / 组装 / 训练分进程	并行生产、共享与磁盘缓存	当前最新实现
batch	训练前全量特征物化	传统训练方式的背景说明	不纳入对比实验

- 系统仍需将图拓扑放入内存；目标是降低全量业务特征的常驻需求。

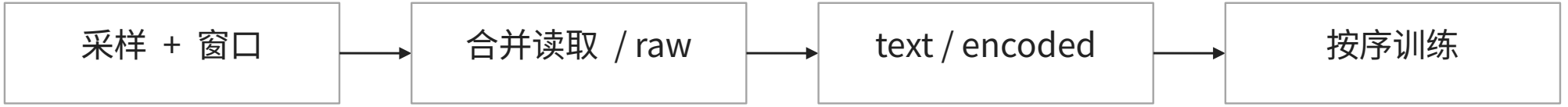
1. 优化路线：减少重复工作 → 提高并行度 → 降低管理成本

vanilla



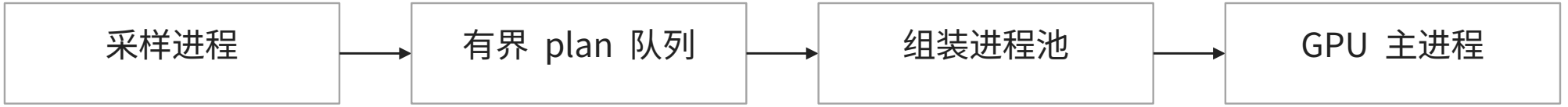
逐 batch 准备；同一节点和文本可能被反复处理。

v1



把相邻请求合并，只处理 miss； SQL 与 encode 线程重叠。

v2

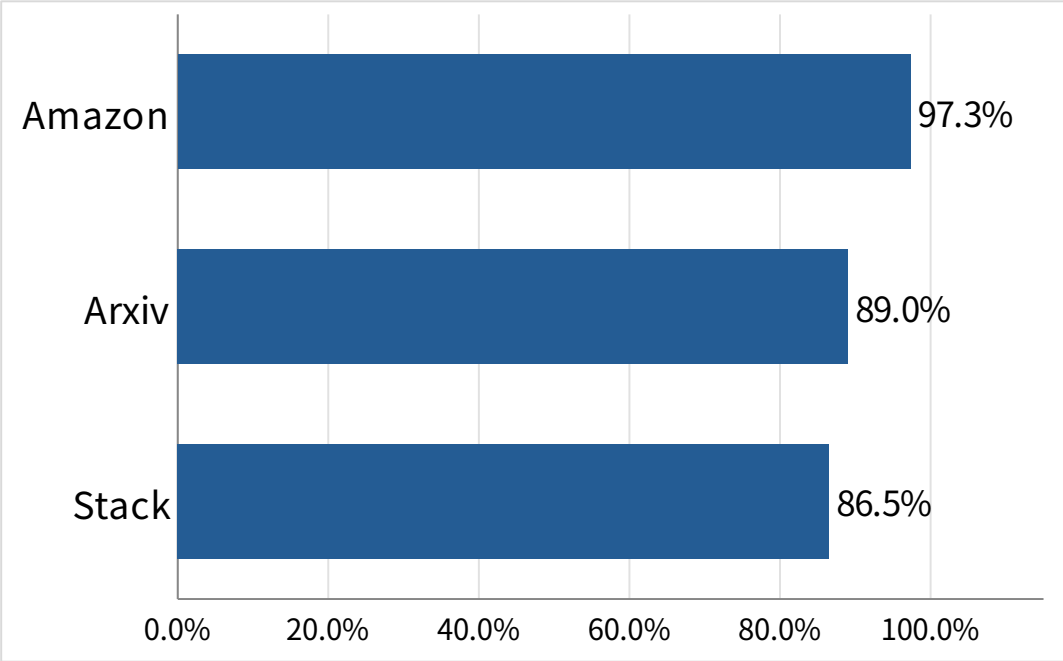


独立进程供给特征；共享结果，按需复用磁盘文本向量。

后续 v2 修复关注： HM 的传输粒度、 Salt/Stack 的共享查询、 mmap 与 WAL 资源增长。

2. 瓶颈定位： GPU 主要在等待 CPU 特征准备

历史 ready 等待占训练时间的比例



特征路径为何昂贵?



- 同一节点被反复采样，重复读取和转换。
- 文本向量未复用时， GloVe 被反复调用。
- 命中缓存后，仍有查询、复制和展开。
- 单个特征 worker 可能无法持续供给 GPU 。

稳态速度由最慢的阶段决定：采样供给、特征生产、 GPU 更新。
初始化、流水线填充与退出，仍会影响端到端时间。

先看 wall time 和 ready 等待，再看阶段计时；并行 / 嵌套操作耗时不能直接相加。

来源： bottleneck-analysis-and-optimization.md ｜历史数据，用于解释优化依据

3. SQL 优化：把扫描变成按 ID 查找，避免过量读取

① rowid 查询：转换 ID ，不改索引键

旧： WHERE rowid − 1 IN (...)
表达式可能破坏直接 rowid 查找

↓

新： WHERE rowid IN (node_id + 1, ...)
ORDER BY rowid
SELECT 输出时再使用 rowid − 1

② 稀疏 / 稠密访问使用不同读取方式

稀疏请求：只取零散 ID ，合并 IN 查询



稠密请求：读取整块，进入 raw cache

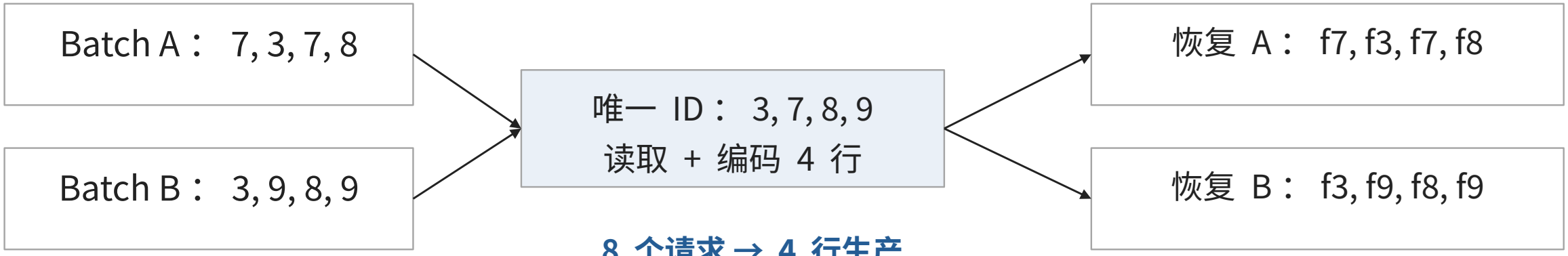


- SQL 放大率关注实际查询行数 / 实际需要查询的行数； featureless 节点须排除。
- raw 命中省下数据库读取和解码，但并不省掉后面的文本 / TensorFrame 工作。
- 当前 vanilla 已回补 rowid 修复；旧版超时不能代表今天的 vanilla 。

收益来自“少扫、少取、少解码”；正式实验需固定数据库映射与修订后的输入语义。

4. 去重与窗口：只做一次特征生产，再恢复训练输入

示例：两个 batch 合并为一个特征窗口（示意数据）

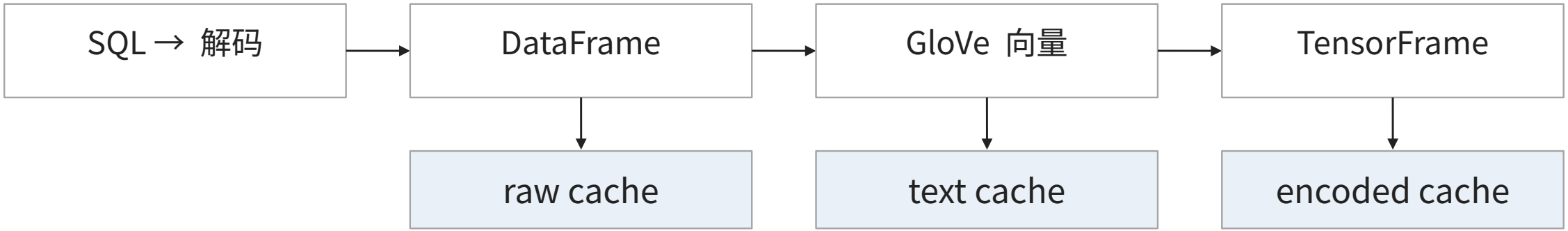


8 个请求 → 4 行生产
训练仍保留 8 个位置

复用范围	省下的工作	必须保留的语义
批内 / 推荐分支	合并同表节点及推荐任务分支	节点重复位置、边顺序、 label
相邻 batch 窗口	合并 SQL 、解码与静态特征编码	分别恢复各 batch 和 seed time

窗口越大不一定越好：受 batch / byte 上限约束；跨 epoch 预取仍须拆成独立窗口。

5. 缓存逐层上移：越接近最终特征，省掉的重复工作越多



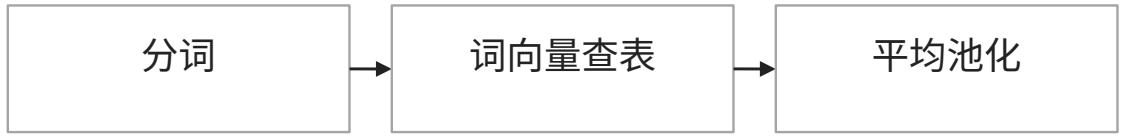
层级	缓存对象	命中可省掉	命中后仍需付出的成本
raw	解码后的 DataFrame 块	SQL / 解码	文本处理、 Frame 转换
text	冻结的 300 维句向量	GloVe 模型执行	查询、 gather 、 Frame 转换
encoded	静态 CPU TensorFrame	重复 prepare / converter	缓存查询、切片与恢复顺序

- 按实际持有的 storage 计费；有界淘汰与准入保护热点，超大项拒绝入缓存。

高命中率不等于低成本；后续 HM 、 Salt 、 Stack 都说明“访问缓存本身”可能成为瓶颈。

6. v1 第二轮：减少通用框架开销，并让 SQL 与编码重叠

文本快路径：保留官方计算语义



- direct 复用官方模块；不匹配时回退 encode 。
- 连续 slab 减少小 Tensor / storage 对象。

线程流水线：按顺序消费

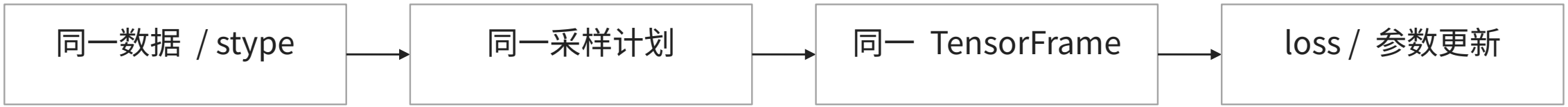


- 窗口按重复率与 raw 命中率自适应。
- schema / topology 持久化降低重复初始化。

已有实验	对照	优化	解释范围
文本微基准（ 20 轮）	0.2123 s	0.1334 s	局部约 1.59× ，非训练加速
Salt GPU 2×50 batch	213.58 s	140.99 s	组合优化，训练耗时 ↓ 34.0%
Ratebeer GPU 2×20	57.77 s	52.83 s	短程训练耗时 ↓ 8.5%

F1 CPU 小任务反而退化约 1.5%；多 worker 的收益取决于特征瓶颈和额外开销。

7. 正确性：先对齐输入，再比较性能

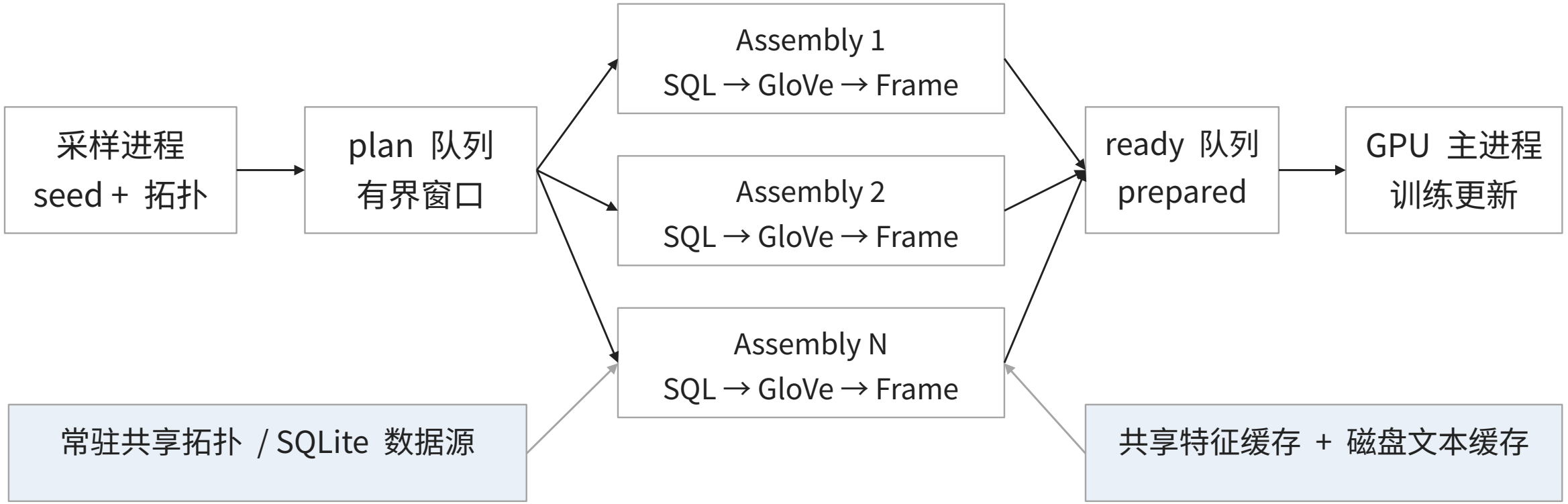


必须固定	具体要求	为什么影响比较
字段与目标	共同读取官方 stype ；按任务隐藏目标列	改型 / 删列会改变编码量和学习问题
结构与时间	节点 / 边顺序、重复项、 seed time	去重只省生产，不能减少训练输入
缓存内容	只保存静态特征与冻结文本向量	训练中的 GNN 表示会随参数更新
执行顺序	strict 固定顺序； ready 单独评估	batch 顺序改变优化器轨迹

- 已有证据：固定 fixture 逐值一致；真实 F1 推荐任务的 60 组特征与结构一致。

修订标识 relbench-correctness-v1 ；修复前后成绩分开，不能把历史 v1 当成当前 v1 。

8. v2 多进程架构：组装进程完成“读取到特征”的完整路径



- worker 内保留 reader、原始字符串和 DataFrame；跨进程传计划与准备好的 tensor。

v1 单进程优化后 assemble 仍受限； v2 将采样、组装与训练分进程重叠执行。

9. 调度： ready 减少队头等待，但训练顺序会改变

时序示意： B2 在 t=1 就绪， B3 在 t=2 就绪， B1 在 t=4 就绪

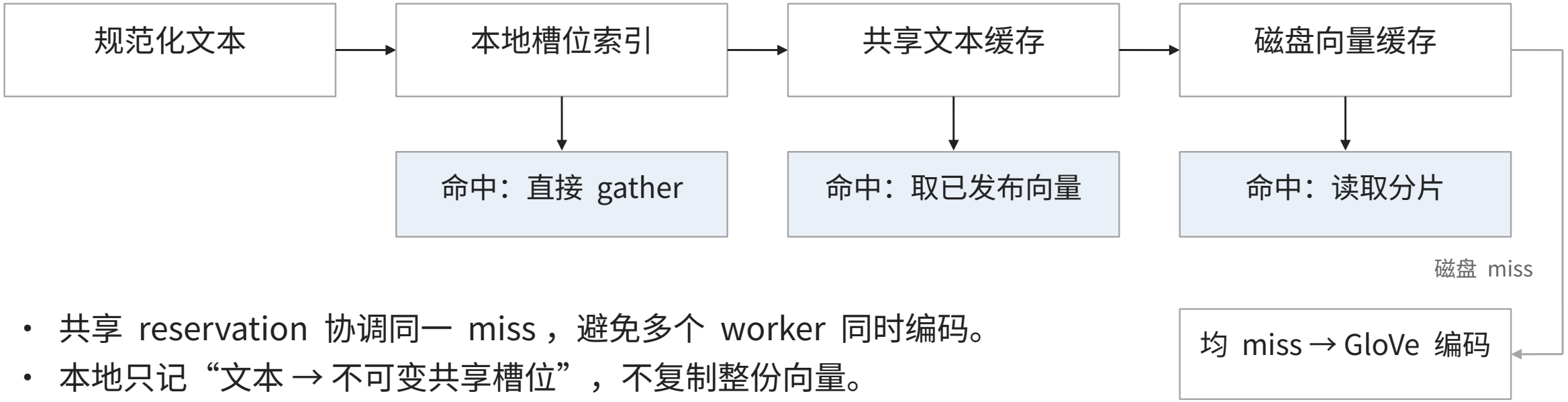


机制	作用	边界
epoch gate	只训练当前 epoch ； 下一轮可提前准备	一个特征窗口不能混合多个 epoch
队列 + inflight	限制排队字节与未完成窗口	超前准备不应无限累积
动态活跃数	ready 饥饿时扩展，高水位时减少	预先 spawn ； 暂停不等于释放内存

strict 用于固定顺序的正确性 / 性能对照； ready 的吞吐收益与收敛影响单独报告。

10. 文本缓存路径：先复用，再编码；高复用时绕过共享锁

向右表示 miss；本地索引只在高收益工作负载下激活。



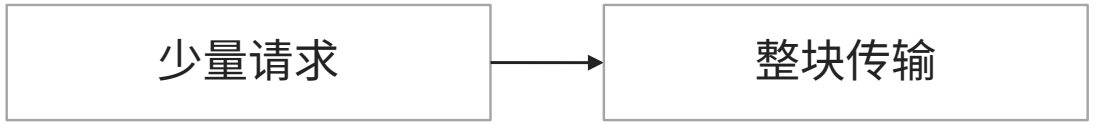
- 共享 reservation 协调同一 miss，避免多个 worker 同时编码。
- 本地只记“文本 → 不可变共享槽位”，不复制整份向量。
- 低复用时释放索引并冷却；每 worker 约 32 MiB 元数据预算。

磁盘 adaptive：数据库 ≥ 256 MiB，模型输入 ≥ 4,096、模型耗时 ≥ 1 s，内存命中率 ≤ 10% 。

索引策略与磁盘策略独立； F1 可不激活， Arxiv 低复用时可自动退出本地索引。

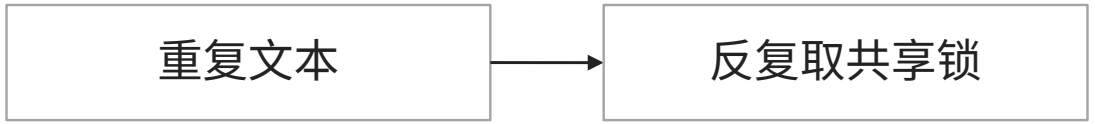
11. HM / Salt： 命中缓存以后，访问成本才是瓶颈

HM： 整块 RPC → 只传行 / 本地缓存



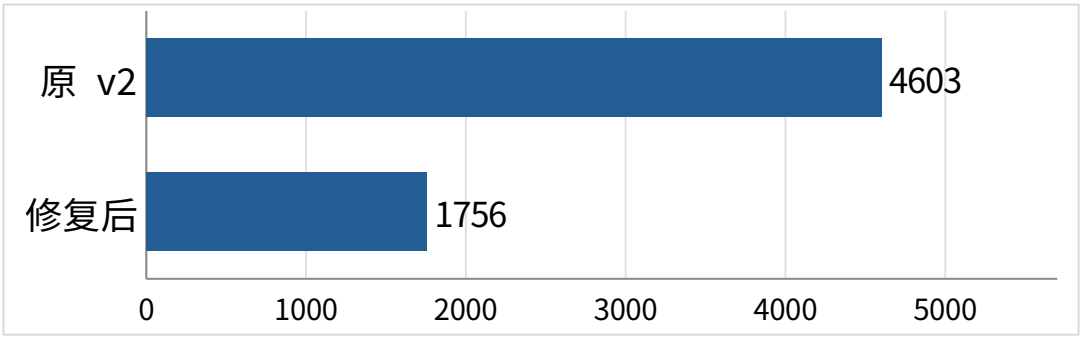
- article 命中率 99.77%，但 RPC 很重。
- 先只返回命中行，再提供 local encoded 。

Salt： 频繁共享查询 → 本地槽位索引



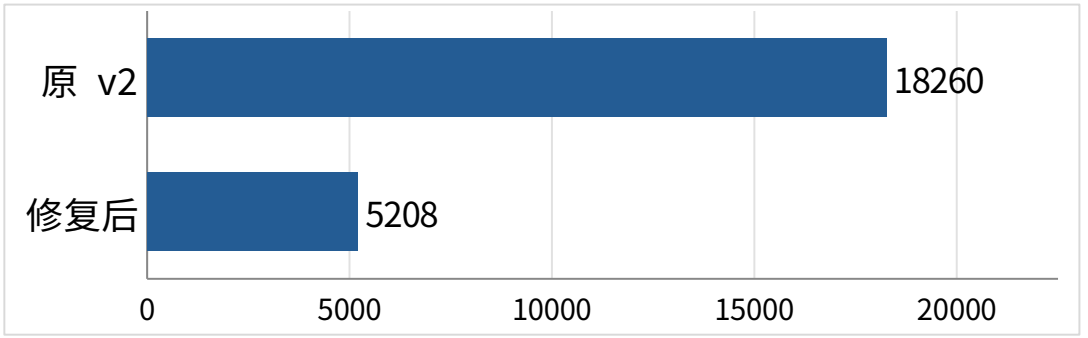
- 共享命中率约 99.93%，第二轮编码为 0 。
- 向量共享不变；重复命中绕过共享锁。

HM： 两轮 pipeline （秒）， 2.62×



shared 2 GiB → local 总计 4 GiB

Salt： 两轮 wall （秒）， 3.51×



完整 6,340 batch，模型输入不变

这些是“修复旧 v2” 的倍数； HM 还改变作用域 / 容量，不能写成对 v1 的单因素提升。

12. Stack / Amazon： 并行训练还需要控制系统资源

Stack： 查询竞争与 mmap 数持续增长

逐槽查询 / clone → 长锁等待

分片映射不关闭 → 映射 / FD 耗尽

- 哈希与批量 gather 移到锁外。
- 每 cache 最多 1,024 个映射， LRU 回收。
- 同步错误通道，避免报错时再建线程。

完整 13,228 batch： 3,665.66 s
越过旧挂起点， worker 正常退出。

Amazon： WAL 增长与 checkpoint 抖动

大量小写入 → WAL 约 60.46 GB

固定 64 MiB 阈值 → 频繁刷盘 / 等锁

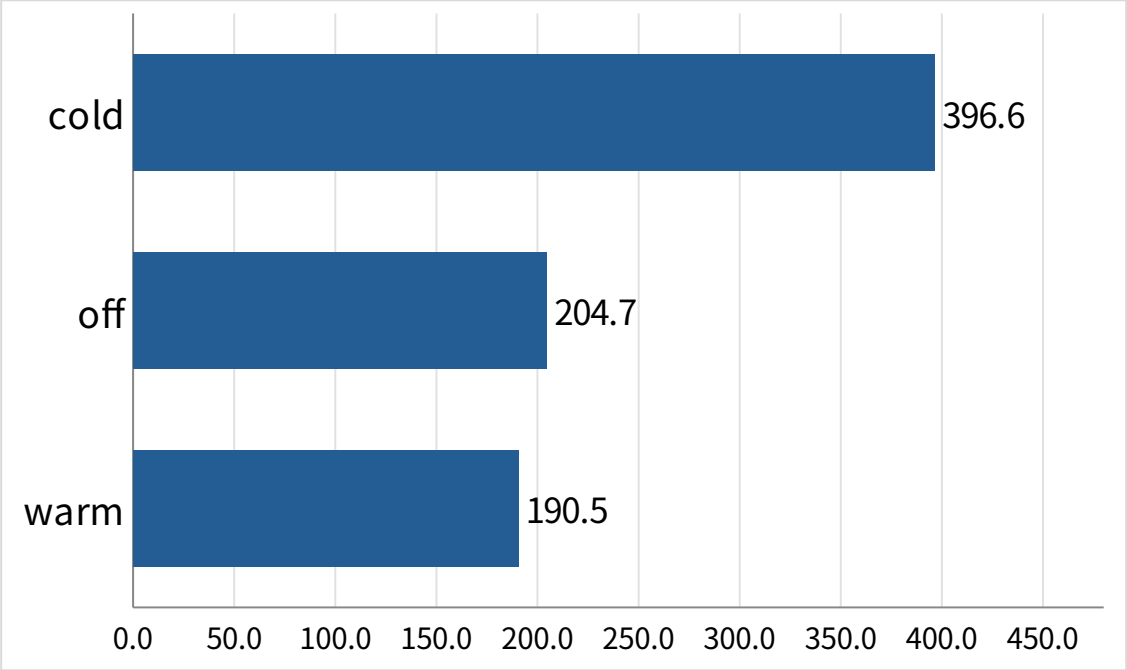
- 每 writer 独占 8,192 行追加分片。
- 按索引大小与剩余空间设置回收阈值。
- 阈值是触发条件，不是 WAL 硬上限。

2×100 batch： 676.01 s （仅短测）
WAL 峰值约 1.35 GiB ，退出归零。

内存有界还不够：文件数、映射数、日志增长与错误传输都影响能否持续完成训练。

13. 磁盘缓存：首轮构建成本与后续复用收益分开看

Arxiv 2×50 batch： wall time （秒）



缓存生命周期

首次遇到文本 → 编码 → 写入

后续 batch / epoch → 索引 → 向量

- warm 比 off 快约 6.9% 。
- 模型输入从 574,119 降到 0 。
- 总时间仍受转换、查询、 gather 限制。

冷启动 396.6 s 包含首次初始化和写入；不能把 cold / warm 的 2.08× 全算成编码收益。

100 万条 300 维 float32 向量约 1.12 GiB ，仅为向量 payload ，不含索引和 WAL 。

mmap LRU 只限制打开的映射，不限制持久化向量总量；需要记录初始 / 最终磁盘占用。

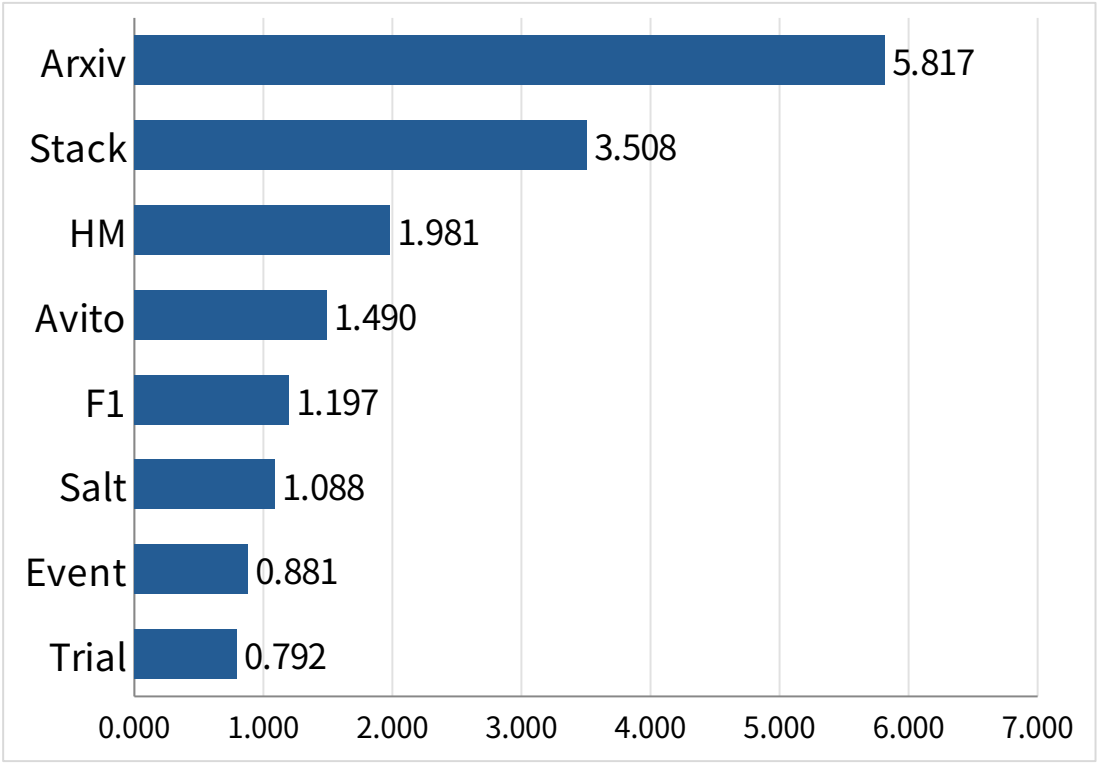
14. 结果总览： 8 项成功记录，仍有退化和未完成任务

以下为报告快照中的最新成功记录；历史 v1 为单轮， v2 为两轮。

任务	batch 数	wall / s	s / batch
Arxiv	2,088	981.64	0.47013
Avito	8,644	1,604.83	0.18566
Event	76	2,193.96	28.86783
F1	12	26.57	2.21388
HM	14,972	1,760.13	0.11756
Salt	6,340	5,208.23	0.82149
Stack	13,228	3,665.66	0.27711
Trial	918	656.95	0.71563

Amazon 、 Ratebeer： 完整结果待补。

历史 v1 / v2 单位 batch 耗时比

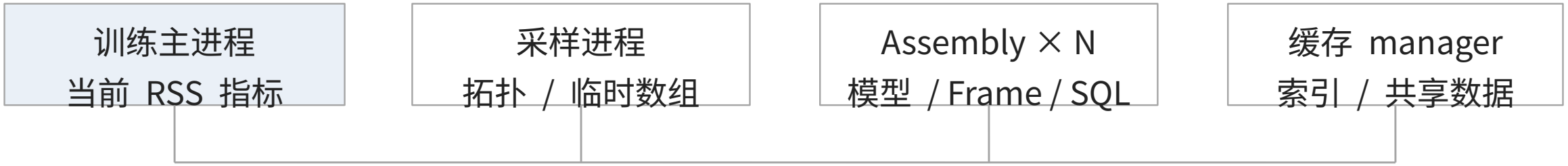


比值 >1： 此组记录中 v2 更快。

硬件、正确性版本、冷热缓存及顺序不同：仅用于观察趋势，不是严格配对加速结论。

15. 内存目标是否达成？必须统计整个进程树

当前 peak_rss_mb = RUSAGE_SELF ，只记录训练主进程，不能代表系统总内存。

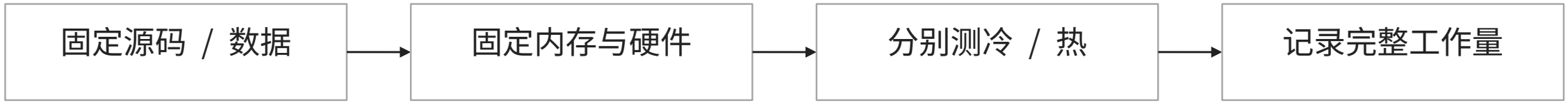


一起测量：进程树 PSS / USS 、共享内存和 cgroup 峰值

配置预算（不是实测）	当前典型值	解释
raw / encoded / text	4 GiB / 2 GiB / 512 MiB	本地缓存按 worker 分摊，共享只计总预算
plan / ready 队列	2 GiB / 16 GiB	容量不等于实际峰值，也非系统硬上限
其他成本	模型、临时对象、在途数据	暂停 worker 不等于归还其全部内存

不能由主进程 2–4 GiB 宣称整个 v2 低内存达标；还需显存、磁盘、 FD / mmap 峰值。

16. 下一轮实验：统一约束，拆开验证每项优化



对照维度	实验设置	回答的问题 / 主要指标
主版本	vanilla / v1 / v2	相同语义与总内存限制下，能否完成、吞吐多少
并行度	1 / 2 / 4 worker ，等总预算	ready 等待、 CPU/I/O 饱和度、进程树内存
encoded 缓存	off / shared / local ，等容量	省编码收益能否覆盖 RPC / 复制成本
调度 / 文本索引	strict vs ready ； shared vs adaptive	顺序对收敛的影响；共享查询成本
磁盘 / 初始化	全冷；初始化暖文本冷；全热	首次成本、多轮摊销、磁盘增长

重复运行并报告波动；短测与完整 epoch 分开，初始缓存不得在后测中默默继承。

优先补 Amazon / Ratebeer ；诊断 Event / Trial ，再用统一最终代码重跑已完成任务。

17. 总结：每一轮优化都在解决新的关键路径问题

阶段	主要改动	解决的问题	仍需关注
vanilla	按需 SQL 读取与在线组装	避免全量业务特征驻留	重复读取 / 编码， CPU 供给不足
v1 第一轮	rowid 修复、去重、窗口、 text	减少扫描与重复特征生产	大文本工作集导致缓存抖动
v1 第二轮	encoded 、 direct 、 slab 、线程	减少重复转换与调度开销	小任务并行开销，线程并行限制
v2	多进程、有界队列、共享 / 磁盘	提高供给与跨轮次复用	RPC 、共享锁与长期资源增长
v2 后续修复	只传行、槽位索引、 mmap/WAL	降低命中成本，保障持续运行	统一最终版本性能与总内存验收

最重要的认识：缓存“命中”只是中间指标，最终要看训练是否更快、资源是否有界。

当前证据支持优化方向；正式结论依赖同语义、同硬件、同内存限额下的配对实验。